

1. **Where state lives:** Where a piece of state lives · Lifting state up: data down, events up · A single source of truth · State belongs to a place in the tree (and key resets it)
2. **Prop drilling and composition:** Prop drilling, and when it actually hurts · Composition: pass components, not data
3. **Context:** Context: a value for a whole subtree · A provider component that owns the state · A custom hook for the context: `useAuth()` · What re-renders when a context value changes · Props, composition or context?
4. **An authentication context:** An authentication context: user, token, log-in, log-out · Staying logged in after a reload · Where to keep the token: the security trade-off · Sharing the token with the API client
5. **Server state and stores:** Server state vs UI state · Data-fetching caches (TanStack Query) · Store libraries: Redux Toolkit and Zustand · Putting it together: a home for every piece of state

Where state lives

Where a piece of state lives

Every piece of state has exactly one **owner**: the component that declares it. Put it in the **closest common parent** of every component that reads or changes it, and no higher.

A task screen: who uses what, and where it lives

Value	Read or changed by	Lives in
text being typed in the new-task form	TaskForm only	TaskForm (local)
search filter	Toolbar (the input) and TaskList	TaskPage, their closest common parent
the list of tasks	Toolbar (count), TaskList, TaskItem (toggle), TaskForm (add)	TaskPage
the logged-in user	Avatar in the header, TaskList, LogoutButton	App, or a context provider near the root
how many tasks match the filter	Toolbar	nowhere: derived during render

Example. In the tool below, choose "Search filter text", then press **Place here** on Toolbar: TaskList is flagged because it is outside Toolbar's subtree. Remove it and place it on App: it works, but Layout and TaskPage pass a prop they never use. Place it on TaskPage: everyone can reach it and nothing is passed on for nothing. Then try the first challenges.

Common mistake. Putting every piece of state at the top "so everything can reach it". It works, but each keystroke in a search box then re-renders the whole app, and every component in between has to pass props it does not care about. Start local and move state up only when a second component needs it.

Lifting state up: data down, events up

Lifting state up means moving a piece of state from a child into a parent so siblings can share it. The parent passes the **value** down as a prop and a **callback** down so children can ask it to change.

```
function TaskPage() {
  const [filter, setFilter] = useState(''); // lifted here
  return (
    <>
      <Toolbar filter={filter} onFilterChange={setFilter} /> /* value + callback */
      <TaskList filter={filter} /> /* value only */
    </>
  );
}

function Toolbar({ filter, onFilterChange }) {
  return <input value={filter} onChange={(e) => onFilterChange(e.target.value)} />;
}
```

```
import { useState } from 'react';
```

```
const TASKS = [
  { id: 1, title: 'Write the API skeleton' },
  { id: 2, title: 'Add the log-in form' },
  { id: 3, title: 'Write the README' },
];

function Toolbar({ filter, onFilterChange, shown }) {
  return (
    <label>
      Search{ ' '}
      <input value={filter} onChange={(e) => onFilterChange(e.target.value)} />
      { ' '}{shown} shown
    </label>
  );
}

function TaskList({ tasks }) {
  return <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}

function TaskPage() {
  const [filter, setFilter] = useState(''); // lifted: both children need it
  const visible = TASKS.filter((t) => t.title.toLowerCase().includes(filter.toLowerCase()));
  return (
    <section>
      <Toolbar filter={filter} onFilterChange={setFilter} shown={visible.length} />
      <TaskList tasks={visible} />
    </section>
  );
}

function App() {
  return <TaskPage />;
}
```

Example. The box lifts the filter into TaskPage. Toolbar is a controlled input that reports each keystroke with onFilterChange; TaskPage derives the visible tasks and passes them to TaskList. Type in the box: both children update together because they read the same state.

Common mistake. Lifting the value but not the change: the parent passes filter down, but the child keeps calling its own local setFilter. The input then shows what it typed while the list filters by the parent's old value. When state moves up, the child must lose its useState and report changes through the callback.

A single source of truth

Each fact the interface shows should be stored **once**. Every other place that needs it receives it (as a prop or from a context) or computes it. Two stored copies of the same fact always end up disagreeing.

```
import { useState } from 'react';

function Counter({ initial }) {
  const [count] = useState(initial); // a second copy of "how many tasks"
  return <p>Header says: {count} tasks</p>;
}

function App() {
  const [tasks, setTasks] = useState(['Buy milk', 'Call Ana']);

  function add() {
    setTasks([...tasks, 'Task ' + (tasks.length + 1)]);
  }

  return (
    <main>
      <Counter initial={tasks.length} />
      <p>List says: {tasks.length} tasks</p>
    </main>
  );
}
```

```

    <button onClick={add}>Add a task</button>
    <ul>{tasks.map((t) => <li key={t}>{t}</li>)}</ul>
  </main>
);
}

```

Example. The box has the bug on purpose: Counter copies `tasks.length` into its own state. Add a task: the list says 3, the header still says 2, because `useState` read its argument only once. Fix it: delete the `useState` line in Counter and render `{count}` from a prop `<Counter count={tasks.length} />`.

Common mistake. Keeping the copies and "syncing" them with an effect: `useEffect(() => setCount(tasks.length), [tasks])`. It renders once with the wrong number, then again with the right one, and every new copy needs another effect. If a value can be read from the owner or computed, it is not state.

State belongs to a place in the tree (and key resets it)

React keeps a component's state as long as the **same component type** is rendered at the **same position** in the tree. Changing its props does not reset its state; giving it a different `key` does.

```

import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Buy milk' },
  { id: 2, title: 'Call Ana' },
];

function EditTask({ task }) {
  const [draft, setDraft] = useState(task.title); // a deliberate copy: the edit in progress
  return (
    <p>
      Editing #{task.id}:{' '}
      <input value={draft} onChange={(e) => setDraft(e.target.value)} aria-label="Title" />
    </p>
  );
}

function App() {
  const [selectedId, setSelectedId] = useState(1);
  const [withKey, setWithKey] = useState(false);
  const task = TASKS.find((t) => t.id === selectedId);

  return (
    <main>
      {TASKS.map((t) => (
        <button key={t.id} onClick={() => setSelectedId(t.id)}>{t.title}</button>
      ))}
      <label>
        <input type="checkbox" checked={withKey} onChange={(e) => setWithKey(e.target.checked)} />
        {' use key={task.id}'}
      </label>
      {withKey ? <EditTask key={task.id} task={task} /> : <EditTask task={task} />}
    </main>
  );
}

```

Example. The box edits the selected task in a form that keeps a draft. Type something, then pick the other task: without a key the old draft stays. Tick the key checkbox and repeat: each task gets a fresh form.

Common mistake. Resetting the form with an effect: `useEffect(() => setDraft(task.title), [task.id])`. The first render after the switch still shows the old draft, then a second render fixes it, and every new field needs another line. `key={task.id}` resets all of the form's state at once, before anything is shown.

Prop drilling and composition

Prop drilling, and when it actually hurts

Prop drilling is passing a prop through components that do not use it, only so it reaches a component further down. A level or two is normal and clear; it hurts when many layers forward many props.

Drilling: fine or painful?

Situation	Verdict
a parent passes task and onToggle to its child	not drilling: the child uses them
one middle component forwards one prop	fine: explicit and easy to follow
a generic Layout forwards user to the header and the page	a smell: try composition
the theme or the user is read in a dozen places across the app	a good case for context

```
function App() {
  const [user, setUser] = useState({ name: 'Ana' });
  return <Layout user={user} onLogout={() => setUser(null)} />;
}
function Layout({ user, onLogout }) {      // uses neither
  return <><Header user={user} onLogout={onLogout} /><TaskPage user={user} /></>;
}
function Header({ user, onLogout }) {      // uses neither
  return <header><Avatar user={user} /><LogoutButton onLogout={onLogout} /></header>;
}
function TaskPage({ user }) {              // uses it only to pass it on
  return <TaskList user={user} />;
}
```

Example. In the state-tree tool, choose "The logged-in user", place it on **App** with Props: the tree marks Layout, Header and TaskPage as "only passes on". That is the challenge "Spot the drilling".

Common mistake. Reaching for context (or a store library) the first time a prop goes through one extra component. Context hides where a value comes from, makes components depend on a provider being present, and can re-render more than you expect. One or two levels of explicit props are easier to read and to test.

Composition: pass components, not data

Instead of passing data through a layout so it can build its content, let the owner of the data **build the content itself** and pass the finished elements in as `children` (or other props). The layout no longer needs to know about the data.

```
import { useState } from 'react';

function Layout({ header, children }) {      // knows nothing about users
  return (
    <div className="layout">
      <header>{header}</header>
      <main>{children}</main>
    </div>
  );
}

function Avatar({ user }) {
  return <strong>{user.name}</strong>;
}

function TaskPage({ user }) {
  return <p>{user.name}'s tasks go here.</p>;
}

function App() {
  const [user, setUser] = useState({ name: 'Ana' });
  const other = user.name === 'Ana' ? 'Leo' : 'Ana';
  const header = (
```

```

    <>
      <Avatar user={user} />{' '}
      <button onClick={() => setUser({ name: other })}>Switch to {other}</button>
    </>
  );
  return (
    <Layout header={header}>
      <TaskPage user={user} />
    </Layout>
  );
}

```

Example. In the box, `Layout` takes a header slot and children; it never sees `user`. `App` owns the user and passes it straight to `Avatar` and `TaskPage`. Switch the user: both update, and `Layout`'s code did not have to change.

Common mistake. Thinking composition means "the `Layout` must render `TaskPage` itself". As soon as `Layout` writes `<TaskPage />` in its own JSX, it is the one that must provide `TaskPage`'s props, and drilling is back. Let the owner of the data write the tag and hand the element over.

Context

Context: a value for a whole subtree

A **context** lets a component provide a value to **every component below it**, at any depth, without passing props. In React: `createContext(default)` makes one, a provider element sets the value for its subtree, and `useContext(SomeContext)` reads it.

```

import { createContext, useContext } from 'react';

const ThemeContext = createContext('light'); // 'light' is the default: no provider above

function Badge() {
  const theme = useContext(ThemeContext); // the nearest provider above wins
  return <span className={'badge ' + theme}>{theme}</span>;
}

function Panel({ title }) { // no theme prop anywhere
  return <p>{title}: <Badge /></p>;
}

function App() {
  return (
    <>
      <Panel title="Outside any provider" />
      <ThemeContext value="dark">
        <Panel title="Inside a dark provider" />
        <ThemeContext value="light">
          <Panel title="Inside a nested light provider" />
        </ThemeContext>
      </ThemeContext>
    </>
  );
}

```

Example. The box has one context and three panels. The first is outside any provider and gets the default; the second is inside a "dark" provider; the third is inside a nested "light" provider, the nearest one. No component receives a theme prop.

Common mistake. Expecting the default value of `createContext` to be shared state. It is only what a component sees when there is **no provider above it**, usually because the provider was forgotten or placed too low. Many teams use `createContext(null)` and a hook that throws, so a missing provider fails loudly instead of quietly using a default (see [A custom hook for the context](#)).

A provider component that owns the state

The usual pattern: one component, for example `ThemeProvider`, declares the state, renders the context provider with the state and the functions that change it as its value, and renders `{children}` inside it.

```
// ThemeContext.jsx
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));
  return <ThemeContext value={{ theme, toggleTheme }}>{children}</ThemeContext>;
}

// main.jsx
createRoot(document.getElementById('root')).render(
  <ThemeProvider><App /></ThemeProvider>
);
```

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));
  return <ThemeContext value={{ theme, toggleTheme }}>{children}</ThemeContext>;
}

function ThemeToggle() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return <button onClick={toggleTheme}>Switch to {theme === 'light' ? 'dark' : 'light'}</button>;
}

function Sidebar() {
  // passes nothing on
  return <aside><ThemeToggle /></aside>;
}

function Page() {
  const { theme } = useContext(ThemeContext);
  return (
    <div className={'page ' + theme}>
      <h2>Tasks</h2>
      <Sidebar />
    </div>
  );
}

function App() {
  return (
    <ThemeProvider>
      <Page />
    </ThemeProvider>
  );
}
```

Example. In the box, `ThemeToggle` sits inside `Sidebar`, which passes nothing on, yet it reads and changes the theme owned by `ThemeProvider`. `Page` reads the same value to choose its colours.

Common mistake. Calling `useContext(ThemeContext)` in the same component that renders the provider and expecting the new value. `useContext` looks **above** the calling component, so the provider's own component never sees its own provider: it already has the state in a variable; use that.

A custom hook for the context: useAuth()

Wrap `useContext` in a custom hook such as `useAuth()` that **throws a clear error** when there is no provider. **Components import the hook, not the context object.**

```
const AuthContext = createContext(null);

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (ctx === null) {
    throw new Error('useAuth must be used inside <AuthProvider>');
  }
  return ctx;
}
```

```
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logIn = (name) => setUser({ name });
  const logOut = () => setUser(null);
  return <AuthContext value={{ user, logIn, logOut }}>{children}</AuthContext>;
}

function useAuth() {
  const ctx = useContext(AuthContext);
  if (ctx === null) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}

function Greeting() {
  const { user, logIn, logOut } = useAuth();
  if (!user) return <button onClick={() => logIn('Ana')}>Log in as Ana</button>;
  return <p>Hello, {user.name} <button onClick={logOut}>Log out</button></p>;
}

// Try: render <Greeting /> outside <AuthProvider> and read the error.
function App() {
  return (
    <AuthProvider>
      <Greeting />
    </AuthProvider>
  );
}
```

Example. The box works as written. Then move `<Greeting />` out of `<AuthProvider>` (put it after the closing tag, inside a fragment `<>...</>`): the preview shows the hook's error message instead of a confusing crash.

Common mistake. Exporting the context object and calling `useContext(AuthContext)` in every component. It works until the first component outside the provider (a test, a storybook page, a new route) receives `null`. A hook gives one place for the check and one name to search for.

What re-renders when a context value changes

When the provider's value changes (compared with `Object.is`), every component that reads that context re-renders, even inside `memo`. A new object literal on every render counts as a change, so memoise the value with `useMemo`.

```
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logIn = useCallback((u) => setUser(u), []);
  const logOut = useCallback(() => setUser(null), []);
  const value = useMemo(() => ({ user, logIn, logOut }), [user, logIn, logOut]);
}
```

```

    return <AuthContext value={value}>{children}</AuthContext>;
  }

import { createContext, useContext, useMemo, useState, memo } from 'react';

const AuthContext = createContext(null);

const UserBadge = memo(function UserBadge() {
  const { user } = useContext(AuthContext);
  console.log('UserBadge rendered');
  return <p>Signed in as {user.name}</p>;
});

function App() {
  const [clicks, setClicks] = useState(0);
  const [user, setUser] = useState({ name: 'Ana' });
  const [stable, setStable] = useState(false);

  const fresh = { user, setUser }; // a new object every render
  const memoised = useMemo(() => ({ user, setUser }), [user]); // same object until user changes
  const value = stable ? memoised : fresh;

  return (
    <AuthContext value={value}>
      <label>
        <input type="checkbox" checked={stable} onChange={(e) => setStable(e.target.checked)} />
        { ' ' }Memoise the value (useMemo)
      </label>
      <p><button onClick={() => setClicks((c) => c + 1)}>Unrelated click: {clicks}</button></p>
      <UserBadge />
    </AuthContext>
  );
}

```

Example. In the box, UserBadge is wrapped in memo and reads the context. Press **Unrelated click** and watch the console: App re-renders, the value object is new, and the badge re-renders although the user did not change. Tick "Memoise the value" and click again: the badge stays quiet.

Common mistake. Putting everything into one big AppContext "to keep it simple". Every consumer then re-renders whenever any part changes: typing in a search field stored there re-renders the header, the sidebar and every list item. Separate contexts for separate concerns, and local state for anything only one area uses.

Props, composition or context?

Use **props** by default, **composition** when a layout is in the way, and **context** for values that many components at different depths need and that change rarely: the logged-in user, the theme, the language.

Choosing how to share a value

Question	If yes
Only one component uses it?	local state in that component
Siblings or a parent and a child need it?	lift it to the closest common parent, pass props
The path goes through wrappers (Layout, Card, Page)?	composition: pass elements as children or slots
Many components at different depths, changes rarely?	context with a provider component
It is a copy of data that lives on a server?	a data-fetching cache (or a custom hook), not context

Example. The state-tree tool flags the cases where context is overkill. Choose "Search filter text", switch to Context and place the provider on TaskPage: the analysis says plain props would do, because nothing sits between TaskPage and the two components that use the filter.

Common mistake. Using context to avoid thinking about where state belongs. A context provider at the root holding the search text, the selected task and the open modal makes the root re-render the whole app's consumers on every keystroke, and hides which screen owns what. Place state first; reach for context for the few values that really are app-wide.

An authentication context

An authentication context: user, token, log-in, log-out

An AuthProvider owns who is logged in: the user and the token in state, an async logIn(email, password) that calls the API and stores both, and logOut() that forgets them. Components read it all with useAuth().

```
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [token, setToken] = useState(null);
  const [user, setUser] = useState(null);

  async function logIn(email, password) {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error); // the form shows it
    setToken(data.token);
    setUser(data.user);
  }
  const logOut = () => { setToken(null); setUser(null); };

  return <AuthContext value={{ user, token, logIn, logOut }}>{children}</AuthContext>;
}

const useAuth = () => useContext(AuthContext);

function LoginForm() {
  const { logIn } = useAuth();
  const [error, setError] = useState('');
  async function handleSubmit(e) {
    e.preventDefault();
    const form = new FormData(e.target);
    try { await logIn(form.get('email'), form.get('password')); } catch (err) { setError(err.message); }
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="email" defaultValue="ana@example.com" aria-label="Email" />
      <input name="password" type="password" placeholder="password123" aria-label="Password" />
      <button>Log in</button> {error && <span role="alert">{error}</span>}
    </form>
  );
}

function Header() {
  const { user, token, logOut } = useAuth();
  if (!user) return <LoginForm />;
  return <p>{user.name} ({user.role}) · token <code>{token}</code> <button onClick={logOut}>Log out</button></p>;
}

function App() {
  return <AuthProvider><Header /></AuthProvider>;
}
```

Example. The box logs in against the mock API. Use ana@example.com (an admin) or leo@example.com (a student) with the password password123; try a wrong password to see the error come back from logIn to the form.

Common mistake. Storing the password, or the whole log-in response "just in case", in the context. Once logged in, the front end needs the token and the public user fields only. A password kept in memory can leak through logs, error reports or dev tools; drop it as soon as the request has been sent.

Staying logged in after a reload

State lives in memory, so a reload logs the user out. To keep the session, the provider **saves the token (and the user) to localStorage when they change and **restores** them once, when it is created.**

```
function AuthProvider({ children }) {
  const [session, setSession] = useState(() => {           // restore once
    try { return JSON.parse(localStorage.getItem('session')) ?? null; }
    catch { return null; }
  });

  useEffect(() => {                                       // save on every change
    try {
      if (session) localStorage.setItem('session', JSON.stringify(session));
      else localStorage.removeItem('session');
    } catch { /* storage blocked: the session lasts until reload */ }
  }, [session]);
  // logIn: setSession({ token: data.token, user: data.user }) · logOut: setSession(null)
  ...
}
```

```
import { createContext, useContext, useEffect, useState } from 'react';

// The preview blocks localStorage, so this falls back to memory. Your app uses localStorage.
const memory = {};
const storage = {
  get: (k) => { try { return localStorage.getItem(k); } catch { return memory[k] ?? null; } },
  set: (k, v) => { try { localStorage.setItem(k, v); } catch { memory[k] = v; } },
  remove: (k) => { try { localStorage.removeItem(k); } catch { delete memory[k]; } },
};

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [session, setSession] = useState(() => {           // restore once, on load
    const saved = storage.get('session');
    console.log(saved ? 'restored the saved session' : 'no saved session');
    return saved ? JSON.parse(saved) : null;
  });
  useEffect(() => {
    if (session) storage.set('session', JSON.stringify(session));
    else storage.remove('session');
  }, [session]);
  const logIn = () => setSession({ token: 'student-token', user: { name: 'Leo' } });
  const logOut = () => setSession(null);
  return <AuthContext value={{ user: session?.user ?? null, logIn, logOut }}>{children}</AuthContext>;
}

function Status() {
  const { user, logIn, logOut } = useContext(AuthContext);
  if (!user) return <p>Logged out <button onClick={logIn}>Log in as Leo</button></p>;
  return <p>Logged in as {user.name} <button onClick={logOut}>Log out</button></p>;
}

function App() {
  const [boot, setBoot] = useState(0);    // a new key = a fresh provider, like a page reload
  return (
    <>
      <button onClick={() => setBoot((b) => b + 1)}>Simulate a reload</button>
    </>
  )
}
```

```

    <AuthProvider key={boot}><Status /></AuthProvider>
  </>
);
}

```

Example. The preview frame blocks `localStorage`, so the box falls back to a variable in memory (your app uses the real storage). Log in, then press **Simulate a reload**: the provider is created again (a new key), its initialiser finds the saved session and you stay logged in. Log out and reload: the session is gone.

Common mistake. Restoring in an effect: `useEffect(() => setToken(localStorage.getItem('token')), [])`. The first render runs with `token = null`, so a protected page redirects to the log-in page before the effect restores the token, and the user lands on the log-in screen although they are logged in. Read storage in the lazy initialiser, before the first render.

Where to keep the token: the security trade-off

A token in **memory** is lost on reload; in **localStorage** it survives but any script on the page can read it; in an **httpOnly cookie** set by the server JavaScript cannot read it at all, but cookies bring their own rules (CSRF, CORS credentials).

Three places for the session

Where	Survives a reload	Readable by any script on the page	Sent automatically	Watch out for
memory (state / context)	no	only while the page is open	no	users logged out on every reload
<code>localStorage</code>	yes	yes	no (you add the header)	XSS steals it
httpOnly cookie	yes	no	yes, by the browser	CSRF, CORS credentials

Example. With a token in `localStorage`, open the browser's dev tools on your own app, Application tab, Local Storage: the token is right there in plain text, and so is it for any script running in the page. With an `httpOnly cookie`, the Cookies panel shows it, but `document.cookie` in the console does not include it.

Common mistake. Believing that `localStorage` is "encrypted" or private to your code, or that `sessionStorage` is safer against XSS. Both are readable by any script running on the page; `sessionStorage` only differs in lasting until the tab closes. The protection against token theft is not the storage name but preventing XSS, and `httpOnly cookies` when that is not enough.

Sharing the token with the API client

The **API client** needs the current token, and a 401 must log the user out. Let the auth provider create the client with the token and its own `logout` as the `onUnauthorized` handler, and expose it through the context.

```

function AuthProvider({ children }) {
  const [session, setSession] = useState(restoreSession);
  const logout = useCallback(() => setSession(null), []);
  const token = session?.token ?? null;
  const api = useMemo(() => createClient({ token, onUnauthorized: logout }), [token, logout]);
  const value = useMemo(() => ({ user: session?.user ?? null, api, logout /* , login */ }), [session, api, logout]);
  return <AuthContext value={value}>{children}</AuthContext>;
}

```

```

import { createContext, useContext, useMemo, useState } from 'react';

function createClient({ token, onUnauthorized }) {
  return async function api(path, { method = 'GET', body } = {}) {
    const headers = { 'Content-Type': 'application/json' };
    if (token) headers.Authorization = 'Bearer ' + token;
    const res = await fetch('/api' + path, { method, headers, body: body && JSON.stringify(body) });
    if (res.status === 401 && token) { onUnauthorized(); throw new Error('Session expired: log in again'); }
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
  };
}

```

```

    return data;
  };
}

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [token, setToken] = useState('expired-token'); // as if restored days later
  const logOut = () => setToken(null);
  const api = useMemo(() => createClient({ token, onUnauthorized: logOut }), [token]);
  async function logIn() {
    const data = await api('/auth/login', { method: 'POST', body: { email: 'ana@example.com', password:
'password123' } });
    setToken(data.token);
  }
  return <AuthContext value={{ token, api, logIn }}>{children}</AuthContext>;
}

function CreateUser() {
  const { token, api, logIn } = useContext(AuthContext);
  const [msg, setMsg] = useState('');
  async function create() {
    try {
      const u = await api('/users', { method: 'POST', body: { name: 'Iris', email: 'iris' + Date.now() +
'@example.com', password: 'longpassword' } });
      setMsg('Created user ' + u.id);
    } catch (err) { setMsg(err.message); }
  }
  return (
    <section>
      <p>Token: <code>{token ?? '(none)'}</code> {!token && <button onClick={logIn}>Log in as Ana</button>}</p>
      <button onClick={create}>Create a user (admins only)</button> <output>{msg}</output>
    </section>
  );
}

function App() {
  return <AuthProvider><CreateUser /></AuthProvider>;
}

```

Example. The box starts with a token "restored from storage days ago" that the server no longer accepts. Press **Create a user**: the client gets 401 with a token, calls the provider's logOut, and the token disappears. Log in as Ana (a real call to the mock API) and create again: 201. Log out and try once more: 401 without a token is just an error, not a log-out.

Common mistake. Reading the token from localStorage inside the client on every request. It looks like a single source, but now there are two: the context says "logged out" while storage still has the token (or the other way round after a failed write), and logging out in one tab behaves differently from another. The provider owns the session; storage is only its backup.

Server state and stores

Server state vs UI state

UI state belongs to the interface (is the menu open, what is typed, which tab is active). **Server state** is a copy of data that lives on a server (the tasks, the users): it can go stale, is shared by many screens and must be refetched after changes.

Which kind is it?

Value	Kind	Where it goes
is the side menu open	UI	local state
the search text	UI	lifted state, or the URL (?search=)
the logged-in user and token	UI (the session)	auth context
the list of tasks from GET /api/tasks	server	a fetching hook or a query cache

Value	Kind	Where it goes
the task being edited (unsaved)	UI (a draft)	local state in the form

```
import { useEffect, useState } from 'react';

// Each call keeps its OWN copy of the server's tasks.
function useTasks() {
  const [tasks, setTasks] = useState([]);
  async function reload() {
    const res = await fetch('/api/tasks');
    setTasks(await res.json());
  }
  useEffect(() => { reload(); }, []);
  return { tasks, reload };
}

function TaskCount() {
  const { tasks } = useTasks(); // request 1
  return <p>Header: {tasks.length} tasks</p>;
}

function TaskList() {
  const { tasks, reload } = useTasks(); // request 2, same URL
  async function add() {
    await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'New task ' + tasks.length }),
    });
    reload(); // refreshes THIS copy only
  }
  return (
    <>
    <p>List: {tasks.length} tasks</p>
    <button onClick={add}>Add a task</button>
    </>
  );
}

function App() {
  return <><TaskCount /><TaskList /></>;
}
```

Example. In the box, the header and the list both call the same `useTasks()` hook. The console shows **two** identical requests. Add a task from the list: the list refetches its copy, the header keeps its old count. Two copies of server state, two truths.

Common mistake. Loading the tasks once into a global context "so every screen has them" and never refetching. The context now holds a snapshot that ages: another user's changes never appear, and each write needs hand-written code to patch the global list. Server data needs a plan for staleness, not just a place to live.

Data-fetching caches (TanStack Query)

A **data-fetching cache** keeps one copy of each server response under a **key** (`['tasks']`), shares it with every component that asks for the same key, removes duplicate requests and refetches when you **invalidate** the key after a write. TanStack Query (formerly React Query) and SWR are the common libraries.

```
// With TanStack Query (wrap the app once in <QueryClientProvider client={queryClient}>)
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function useTasks(search) {
  const { api } = useAuth();
  return useQuery({ queryKey: ['tasks', { search }], queryFn: () => api('/tasks?search=' +
    encodeURIComponent(search)) });
}
```

```
function useAddTask() {
  const { api } = useAuth();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (title) => api('/tasks', { method: 'POST', body: { title } }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['tasks'] }), // every tasks list refetches
  });
}

// in a component
const { data: tasks, isPending, isError, error } = useTasks(search);
```

```
import { useEffect, useState } from 'react';

// A toy query cache: one entry per key, shared by every component that asks for it.
const cache = new Map(); // key → { data, promise, listeners }
function entry(key) {
  if (!cache.has(key)) cache.set(key, { data: undefined, promise: null, listeners: new Set() });
  return cache.get(key);
}
function fetchKey(key, fn) {
  const e = entry(key);
  if (!e.promise) e.promise = fn().then((data) => { e.data = data; e.promise = null; e.listeners.forEach((l) => l(data)); });
  return e.promise;
}
function useQuery(key, fn) {
  const e = entry(key);
  const [data, setData] = useState(e.data);
  useEffect(() => {
    e.listeners.add(setData);
    if (e.data === undefined) fetchKey(key, fn); // first asker fetches, the others share the promise
    return () => e.listeners.delete(setData);
  }, [key]);
  return data;
}

const getTasks = () => fetch('/api/tasks').then((r) => r.json());

function TaskCount() {
  const tasks = useQuery('tasks', getTasks);
  return <p>Header: {tasks ? tasks.length : '...'} tasks</p>;
}

function TaskList() {
  const tasks = useQuery('tasks', getTasks);
  async function add() {
    await fetch('/api/tasks', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify({ title: 'New task' }) });
    fetchKey('tasks', getTasks); // "invalidate": refetch once, notify everyone
  }
  return <><p>List: {tasks ? tasks.length : '...'} tasks</p><button onClick={add}>Add a task</button></>;
}

function App() {
  return <><TaskCount /><TaskList /></>;
}
```

Example. The box is the previous card's app with a 20-line toy cache. The console shows **one** request although two components ask for the tasks. Add a task: the cache refetches once and **both** components update.

Common mistake. Copying the query's data into `useState` (`const [tasks, setTasks] = useState(data)` and an effect to sync it). That recreates the second source of truth the cache exists to remove: the copy misses every refetch. Read data from the hook directly; for edits, keep a separate draft.

Store libraries: Redux Toolkit and Zustand

A **store** is an object outside the component tree that holds app-wide state; components **subscribe to the slice they select** and re-render only when that slice changes. Redux Toolkit and Zustand are common libraries; most apps of moderate size do not need one.

```
// Zustand
import { create } from 'zustand';
export const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
}));
const count = useCart((s) => s.items.length);    // re-renders only when the count changes

// Redux Toolkit
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: { add: (state, action) => { state.items.push(action.payload); } }, // Immer makes this immutable
});
const count = useSelector((s) => s.cart.items.length);
dispatch(cartSlice.actions.add(item));
```

```
import { useSyncExternalStore } from 'react';

function createStore(initial) {
  let state = initial;
  const listeners = new Set();
  return {
    get: () => state,
    set: (change) => { state = { ...state, ...change(state) }; listeners.forEach((l) => l()); },
    subscribe: (l) => { listeners.add(l); return () => listeners.delete(l); },
  };
}

const store = createStore({ count: 0, theme: 'light' });
const useStore = (selector) => useSyncExternalStore(store.subscribe, () => selector(store.get()));

function Count() {
  const count = useStore((s) => s.count);
  console.log('Count rendered');
  return <p>count: {count}</p>;
}

function Theme() {
  const theme = useStore((s) => s.theme);
  console.log('Theme rendered');
  return <p>theme: {theme}</p>;
}

function App() {
  // reads nothing, so it never re-renders
  return (
    <>
      <Count />
      <Theme />
      <button onClick={() => store.set((s) => ({ count: s.count + 1 })))>+1</button>{' '}
      <button onClick={() => store.set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })))>Theme</button>
    </>
  );
}
```

Example. The box builds a 10-line store with `useSyncExternalStore`. Count selects count, Theme selects theme. Press **+1**: only Count logs a render. Press **Theme**: only Theme does. With one context holding both, both would re-render every time.

Common mistake. Moving every piece of state into the store, including form fields and whether a dropdown is open. The store then becomes a global variable with extra steps: components depend on it for things only they use, tests need it set up, and you lose the "state lives where it is used" rule. Keep local state local even when a store exists.

Putting it together: a home for every piece of state

For each value ask, in order: is it derived (compute it), used by one component (local), by a few nearby (lift it), part of the URL (put it there), app-wide and slow-changing (context), or server data (a fetching hook or cache)? A store is for what is left.

A home for every value in a task app

Value	Home
number of done tasks	derived during render
the new-task title being typed	local state in the form
the selected task in a list + detail split	lifted to the page (or <code>/tasks/:id</code> in the URL)
search text and page number	the URL query string, or lifted state
the logged-in user and token	an auth context, saved to storage
the theme	a theme context, saved to storage
the tasks from the server	a fetching hook per resource or a query cache
a complex client-side editor document	a store library

Example. Try the challenges of the state-tree tool in order: keep local state local, lift the filter, remove a duplicate, spot the drilling, fix it with composition, move the theme into a context, then predict the re-renders with and without `memo` and on log-out.

Common mistake. Deciding the "state management architecture" before writing the screens. The right home for a value comes from who uses it, which you only know once the components exist. Start with local state, lift when needed, and introduce a context, cache or store when a concrete pain (drilling, duplicate requests, too many re-renders) shows up.